Oppgave 1 – modellering (25%, ~1 time)

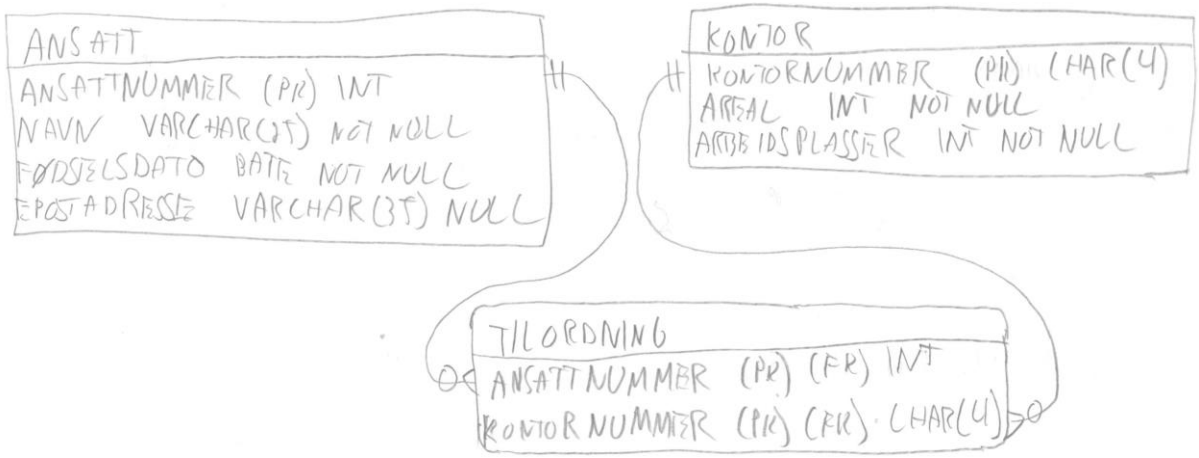
En bedrift har behov for å holde styr på hvilke ansatte som sitter på hvilke kontor. En ansatt har navn, fødselsdato, epostadresse, samt et unikt ansattnummer. Et kontor har areal, antall arbeidsplasser og et unikt kontornummer. Selv om de fleste ansatte nok vil være tilordnet (sitter på) et kontor, må det også være mulig å ikke ha noe kontor, eller å ha flere kontorer. Tilsvarende, selv om de fleste kontor nok vil være tilordnet en ansatt, må det også være mulig å ikke ha noen ansatt (ubrukt), eller å ha flere ansatte (delt).

Lag et logisk (fullstendig) ER-diagram for denne databasen. Ikke glem å ha med:

- Tabeller (som tilfredsstiller alt i problembeskrivelsen)
- Primærnøkler (for alle tabeller)
- Fremmednøkler (for alle relevante tabeller)
- Datatyper (for alle tabeller)
- Både min og maks kardinalitet (for alle forhold)
- Sterke/svake entitetstyper og eksistensavhengighet/uavhengighet (kråkefot-notasjon) eller eventuell type aggregering (UML-notasjon)
- Tilfredsstiller 1., 2., og 3. normalform (for alle tabeller)

Dersom du ikke er sikker på hvordan du skal løse en oppgave, forsøk å gjøre så mye som du kan!

Løsningsforslag



Oppgave 2 (25% ~1 time) – SQL

Oppgaven baserer seg løst på problembeskrivelsen til oppgave 1. Anta at en forenklet/ redusert/ minimal løsning kan bestå av disse to tabellene:
Ansatt(ansattnummer, navn)
Tilordning(ansattnummer*, kontornummer)

1. Skriv SQL-kode for å opprette tabellen Ansatt. Du kan anta at tabellen Tilordning allerede eksisterer. Husk primærnøkkel, eventuelle fremmednøkler/ referanseintegriteter, datatyper, null/ not null, osv.
2. Skriv SQL-kode for å legge til en ny kolonne Telefonnummer til tabellen Ansatt.
3. Skriv SQL-kode for å lage indeks på kolonnen Telefonnummer i tabellen Ansatt.
4. Skriv SQL-kode for å fjerne kolonnen Telefonnummer fra tabellen Ansatt.
5. Skriv SQL-kode som viser ansattnummer for alle ansatte som sitter på kontor E408.
6. Skriv SQL-kode som viser navn for alle ansatte som sitter på kontor E408.
7. Skriv forrige delspørsmål som relasjonsalgebra.
8. Skriv SQL-kode som viser antall ansatte som sitter på hvert kontor.
9. Skriv SQL-kode som viser hvilke kontor det sitter flest ansatte på.
10. Skriv SQL-kode som viser hvilke ansatte som ikke sitter på noe kontor.

Dersom du ikke er sikker på hvordan du skal løse en oppgave, forsøk å gjøre så mye som du kan!

Løsningsforslag

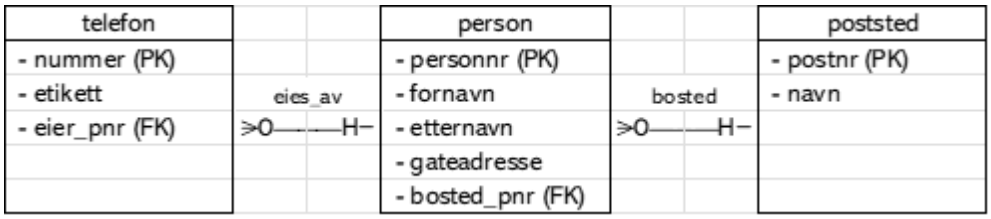
1. Create table ansatt(
   ansattnummer int primary key,
   navn varchar(25) not null);
2. Alter table ansatt add column telefonnummer varchar(35) null;
3. Create index telefonnummerindex on ansatt(telefonnummer);
4. Alter table ansatt drop column telefonnummer;
5. Select ansattnummer
   from tilordning;
   where kontornummer='E408';
6. Select navn
   from ansatt inner join adgang
   on ansatt.ansattnummer=tilordning.ansattnummer
   where kontornummer='E408';
   Eller:
   Select navn
   from ansatt, tilordning
   where ansatt.ansattnummer=tilordning.ansattnummer
   and kontornummer='E408';
7. $\Pi_{navn}\sigma_{kontornummer='E408'}(ansatt \bowtie_{ansatt.ansattnummer=tilordning.ansattnummer} tilordning$
8. Select kontornummer, count(*)
   from tilordning
   group by kontornummer;

9. Select kontornummer, count(*)
   from tilordning
   group by kontornummer
   order by count(*) desc;
10. Select *
    from ansattt as a
    where not exists
    (select * from tilordning as t where a.ansattnummer=t.ansattnummer);
    Eller:
    Select *
    from ansatt
    where ansattnummer not in
    (select ansattnummer from tilgang);
    Andre muligheter finnes også.

Oppgave 3 (25% ~1 time) – ORM/JPA

Vi skal jobbe litt med en enkel database for **person**er. Siden en person kan ha flere **telefon**er, har vi en egen tabell for disse. Siden postnr og navn for et **poststed** henger sammen, har vi en egen tabell for disse.

Her er den logiske ER-modellen for databasen:



Som du ser, er det et **en-til-mange-forhold mellom person og telefon**. Dette skal forstås som at en person kan ha null til mange telefoner, men at hver telefon **eies av** én person.

Som du ser, er det et **en-til-mange-forhold mellom poststed og person**. Dette skal forstås som at et poststed kan ha null til mange personer som bor der, men at hver person **har bosted** tilhørende ett poststed.

PK angir primærnøkler, og **FK** angir fremmednøkler. Her er tabeller med noen eksempeldata:

telefon

nummer (PK)	etikett	eier_pnr (FK)
...		
87654321	'mobil'	'01010155555'
76543210	'jobb'	'01010155555'
65432109	'mobil'	'02020266666'
...		

person

personnr (PK)	fornavn	etternavn	gateadresse	bosted_pnr (FK)
...				
'01010155555'	'Per'	'Persen'	'Heien 1'	5037
'02020266666'	'Ole'	'Olesen'	'Kirkegaten 1'	5036
'03030377777'	'Åse'	'Åsesen'	'Kirkegaten 10'	5036
...				

poststed

postnr (PK)	navn
...	
5036	'Bergen'
5037	'Bergen'
...	

Alle data er av typen VARCHAR, med unntak av telefonnr og postnr, som er av typen INTEGER. Man kan godt argumentere for at databasen burde vært bedre normalisert, men dere får bruke det oppsettet som er beskrevet.

Vi ønsker å jobbe med denne databasen i et Java-program.

a) (9% ~ 20 min) Skriv Java-klassene for entitetstypene `Poststed`, `Telefon` og `Person` som tilsvarer tabellene i eksempelet vist over. Du trenger ikke å skrive metoder, kun instans-/objektvariabler og JPA-annoteringer. Husk de nødvendige annoteringer for å angi forholdet mellom entitetene:

- Vi **ønsker** at en **Person** skal inneholde en **liste over sine telefoner**, slik at vi ved behov enkelt kan få ut disse sammen med andre personopplysninger.
- Vi ønsker **IKKE** at et **Poststed** skal ha en **liste over alle personer som bor der**. Hvis det er behov for en liste over personer, må dette hentes via en spørring.

```
Løsningsforslag:

1 @Entity
  public class Poststed {
2     @Id private int postnr;
private String navn;
...
  }

  @Entity
  public class Telefon {
@Id private int nummer;
private String etikett;
```

```
1       @ManyToOne @JoinColumn(name="eier_pnr")
1       private Person eier;
...
  }

  @Entity
  public class Person {
1       @Id private String personnr;
private String fornavn;
private String etternavn;
private String gateadresse;

1       @ManyToOne @JoinColumn(name="bosted_pnr")
1       private Poststed bosted;

1       @OneToMany(mappedBy="eier", fetch=FetchType.EAGER)
1       List<Telefon> telefoner;
...
  }
```

Du kan i de videre oppgavene anta at entitetsklassene inneholder de nødvendige konstruktører, gettere og settere, etc. du trenger i løsningene dine.

Vi antar at vi har en hjelpeklasse **PersonDAO** for databaseoperasjoner. Du skal lage et par metoder i denne. Du kan anta at en EntityManagerFactory kalt emf er opprettet og tilgjengelig for bruk i metodene.

b) (4% ~ 10 min) Skriv en metode

```
Person finnPerson(String personnr)
```

i PersonDAO som henter ut personen med gitt personnr. Hvis ingen personer med dette personnr finnes, skal det returneres null.

```
Løsningsforslag:

    Person finnPerson(String personnr) {

1        EntityManager em = emf.createEntityManager();

1        try {
5            return em.find(Person.class, personnr);
1        } finally {
1            em.close();
        }
    }
```

c) (4% ~ 15 min) Skriv en metode

```
List<Person> finnPersonerMedPostnr(int postnr)
```

i PersonDAO som henter ut alle personer som bor på gitt postnr. Tips: Her må det skrives en JPQL-spørring.

```
Løsningsforslag:

    List<Person> finnPersonerMedPostnr(int postnr) {

        EntityManager em = emf.createEntityManager();

3       String q = " select p from Person as p where p.bosted.postnr = :postnr";

        try {

2            TypedQuery<Person> query = em.createQuery(q, Person.class);
2            query.setParameter("postnr", postnr);
2            return query.getResultList();

        } finally {
1           em.close();
        }
    }
```

d) (8% ~ 15 min) Skriv en metode

```
boolean leggTilTelefon(

         String personnr, int telefonnr, String etikett)
```

i PersonDAO som legger til en telefon for personen med gitt personnr. Hvis ingen person finnes i databasen med dette personnr, eller det allerede finnes en telefon i databasen med gitt telefonnr, kan ikke telefonen legges til, og det skal returneres **false**. Ellers skal telefonen legges til, og det returneres **true**.

```
Løsningsforslag:

    boolean leggTilTelefon(String personnr, int telefonnr, String etikett) {

        EntityManager em = emf.createEntityManager();
        EntityTransaction tx = em.getTransaction();
        try {
1            Person person = em.find(Person.class, personnr);

             //Hvis person ikke finnes ...
```

```
1           if (person == null) {
                System.err.println("Person med personnr " + personnr +
                        " finnes ikke.");
                return false;
            }
            //Hvis telefon allerede finnes ...
1           if (em.find(Telefon.class, telefonnr) != null) {
                System.err.println("Telefon med telefonnr " + telefonnr +
                        " finnes allerede.");
                return false;
            }
1           tx.begin();

            //Opprette telefon-objekt
2           Telefon telefon = new Telefon(telefonnr, etikett, person);

            //Husk å oppdatere reverse-side av forholdet
1           person.leggTilTelefon(telefon);

            //Lagre i databasen
2           em.persist(telefon);

1           tx.commit();
            return true;
        } finally {
            em.close();
        }
    }
```

Oppgave 4 (25% ~60 time) – XML, JSON og NoSQL

4.1 XML, XPath og JSON (ca 8,5% ~ 20 min)

I oppgavene under (a-e) skal du skrive XPath-uttrykk basert på XML-dokumentet i vedlegg 4.1 (tvguide.xml).

Eksempel:
Oppgave: Skriv et XPath-uttrykk for å finne teksten i alle program beskrivelser.
Svar: /tvguide/program/beskrivelse/text()

Tips:
Når oppgaven inneholder teksten "all informasjon om alle programmer" betyr at du skal hente de aktuelle elementene for de programmene som omfattes av spørringen.

a) Skriv et XPath-uttrykk for å hente alle «program»-elementer (med innhold).
Svar:
/tvguide/program
//program

b) Skriv et XPath-uttrykk for å finne all informasjon om alle programmer i kategorien «Reality».

Svar:
/tvguide/program[kategori='Reality']
//program[kategori='Reality']

c) Skriv et XPath-uttrykk for å finne all informasjon om alle programmer som sendes på kanalen «TV2».

Svar:
/tvguide/program[@kanal='TV2']
//program[@kanal='TV2']

d) Skriv et XPath-uttrykk for å finne titlene på alle programmer i kategorien «Krim».

Svar:
/tvguide/program[kategori='Krim']/tittel/text()

e) Skriv et XPath-uttrykk for å finne all informasjon om alle programmer som går i reprise.

Mulige Svar:
/tvguide/program[reprise]
/tvguide/program/reprise/..

I siste del av oppgaven (f) skal du skrive om XML filen (vedlegg 4.1) til et JSON dokument (tvguide.json). **NB! Du skal KUN ta med de to første «program» elementene (dvs. id=10211 og id=10212).**

Tips!
Du trenger ikke å skrive hele teksten for «beskrivelse» (Bruk: «Bla bla»).

f) Skriv om XML dokumentet (tvguide.xml) til et JSON dokument (tvguide.json).

Svar:

```
[
  {
    "id": 10211,
    "kanal": "TV2",
    "dato": "2026-01-10",
    "tid": "20:00",
    "tittel": "Farmen",
    "kategori": "Reality",
    "beskrivelse": "Bla bla",
    "episode": "2025-1",
    "reprise": true
  },
  {
    "id": "10212",
    "kanal": "TV2",
    "dato": "2026-01-10",
    "tid": "21:00",
    "tittel": "Nyhetene",
```

```
    "kategori": "Nyheter",
    "beskrivelse": "Bla bla."
  }
]
```

(Oppgavene fortsetter på neste side)

4.2 XML og XSLT (8,5% ~ 20 min)

I oppgavene under (a og b) skal du svare på spørsmål relatert til XSLT-filen i vedlegg 4.2 (tvguide.xslt).

XSLT-filen benytter `<xsl:param ... />` for å gi en verdi til en parameter «valgtDato»:
`<xsl:param name="valgtDato" select="'2026-01-10'"/>`

a) Når du åpner filen tvguide.xml i en nettleser, hvordan vet nettleseren at filen skal transformeres og hvilken XSLT-fil som benyttes for å utføre transformasjonen?

Svar:
Det er angitt i XML-filen med en prosesseringsinstruksjon:
`<?xml-stylesheet type="text/xsl" href="tvguide.xslt"?>`
Deretter er det innholdet i tvguide.xslt som styrer selve transformasjonen.
Dette blir beskrevet nærmere i svaret i neste oppgave.

b) Forklar kort hva som vil vises når du åpner tvguide.xml i en nettleser (dvs. når XML filen er transformert av stilarket tvguide.xslt). Forklar hvorfor dette skjer.

Svar:
Stilarket definerer at output skal være en HTML-side: `<xsl:output method="html" />`

Stylesheet elementet `<xsl:template match="/tvguide">` sørger for at transformasjonen returner basis tagene for en HTML side for root elementet "tvguide".
Template taggen vil gjøre at innholdet I taggen gjentas for alle matchende elementer. Men ettersom "tvguide" taggen er root elementet, vil dette kun gjengies en gang.

Dette gjør at man det først vil vises en overskrift "TV-programmer" på siden.
Så kommer en ny overskrift med "Dato: <dato>", der datoen hentes fra parameteren "valgtDato" ved hjelp av `xsl:value-of` taggen.

Det vises videre en tabell over alle programmer som har dato "2026-01-10" (gitt ved parameter). Tabellen inneholder en header rad for kolonnene: Kanal, Tid, Tittel og Kategori. Deretter presenteres alle rad data ved at XSLT-filen filtrerer med `xsl:for-each select="program[@dato=$valgtDato]",` og genererer HTML og tabell rader basert denne løkken, samt diverse kall til de ulike attributter og elementer i de valgte "program" elementene. For å hente verdier fra program attributter brukes f.eks. `<xsl:value-of select="@kanal"/>`.
Mens for å hente verdier fra program sub-elementer benyttes f.eks. `<xsl:value-of select="tittel"/>`.

4.3 NoSQL (ca 2% ~ 5 min)

Nedenfor er det listet 6 databaser (ulike NoSQL-databaser og relasjons databaser), samt en fakta setning om hver av de 6 databasene.

Databaser:	Ulike fakta om ulike databaser:
- MongoDB - Amazon Dynamo DB - PostgreSQL - Cassandra - Neo4j - Oracle	1) Avansert, åpen kildekode-relasjonsdatabase med støtte for SQL og utvidelser som JSON og GIS. 2) En sky-basert NoSQL nøkkel/verdi database (kun skybasert), som bruker et propritært API basert på JSON. 3) Graf database som er optimalisert for å lagre og spørre relasjoner mellom data. 4) En kolonne basert NoSQL database som egner seg godt for analyser og aggregering på store datamengder. 5) En dokument database med høy skalerbarhet, brukes primært for for lagring og håndtering av ustrukturerte eller semi-strukturerte data, som JSON-dokumenter, 6) Kommersiell relasjonsdatabase med omfattende enterprise-funksjoner, høy ytelse og robust sikkerhet.

Koble hver database med EN fakta setning. Svar med <database> = <fakta nr.>. Eksempel: Oracle = 6.

Oppgave: Fullfør koblingene mellom databasene og fakta setningene.

Svar:

```
(Gitt i oppgave: Oracle = 6)
MongoDB = 5
Amazon DynamoDB = 2
PostgreSQL = 1
Cassandra = 4
Neo4j = 3
```

4.4 MongoDB (ca 6% ~ 15 min)

Et nytt TV selskap, «Latter TV», har etablert en MongoDB-database med samlingen «tvguide» i databasen «latter-tv». Vi tenker oss at data fra tvguide.xml er lagt inn i samlingen som JSON dokumenter med nøklene:

- _id (objectId – generert av MongoDB databasen)
- kanal (tekst)
- dato (tekst på format YYYY-MM-DD)
- tid (tekst på format HH24:MI)
- kategori (tekst)
- tittel (tekst)
- beskrivelse (tekst)

I oppgavene under skal du ta utgangspunkt i at du har en åpen sesjon, og er koblet opp mot riktig database.

Bruk MongoDB Query API-et til å lage metode kall mot MongoDB databasen.

Eksempel:
Oppgave: Lag et metodekall som oppdater beskrivelsen for alle nyhets programmer på TV2 kanalen til «Nytt fra inn- og utland.» i «tvguide» samlingen.

Svar:
```
db.tvguide.updateMany(
    { kanal: "TV2", kategori: "Nyheter"},
    { $set: { beskrivelse: "Nytt fra inn- og utland" }
);
```

a) Lag et metodekall mot «tvguide» samlingen som legger til et nytt TV program den 11.januar 2026 på kanalen «NRK» (Finn på verdier for de andre nøklene selv). La MongoDB databasen legge til et unikt «_id» felt for JSON dokumentet som legges til databasen.

Svar:
```
db.tvguide.insertOne(
    {
        kanal: "NRK",
        dato: "2026-01-11",
        tid: "20:00",
        tittel: "2000 dager ute",
        kategori: "Natur",
        beskrivelse: "Bla Bla.",
        episode: "2025-10",
        reprise: true
    }
)
```

b) Lag et metodekall mot «tvguide» samlingen som henter alle programmer med tittelen «Farmen».

Svar:
```
db.tvguide.find({ tittel: "Farmen" })
```

c) Lag et metodekall som henter **antall** programmer som sendes på kanalen «NRK» den 10.januar 2026 (Format: 2026-01-10) fra «tvguide» samlingen.